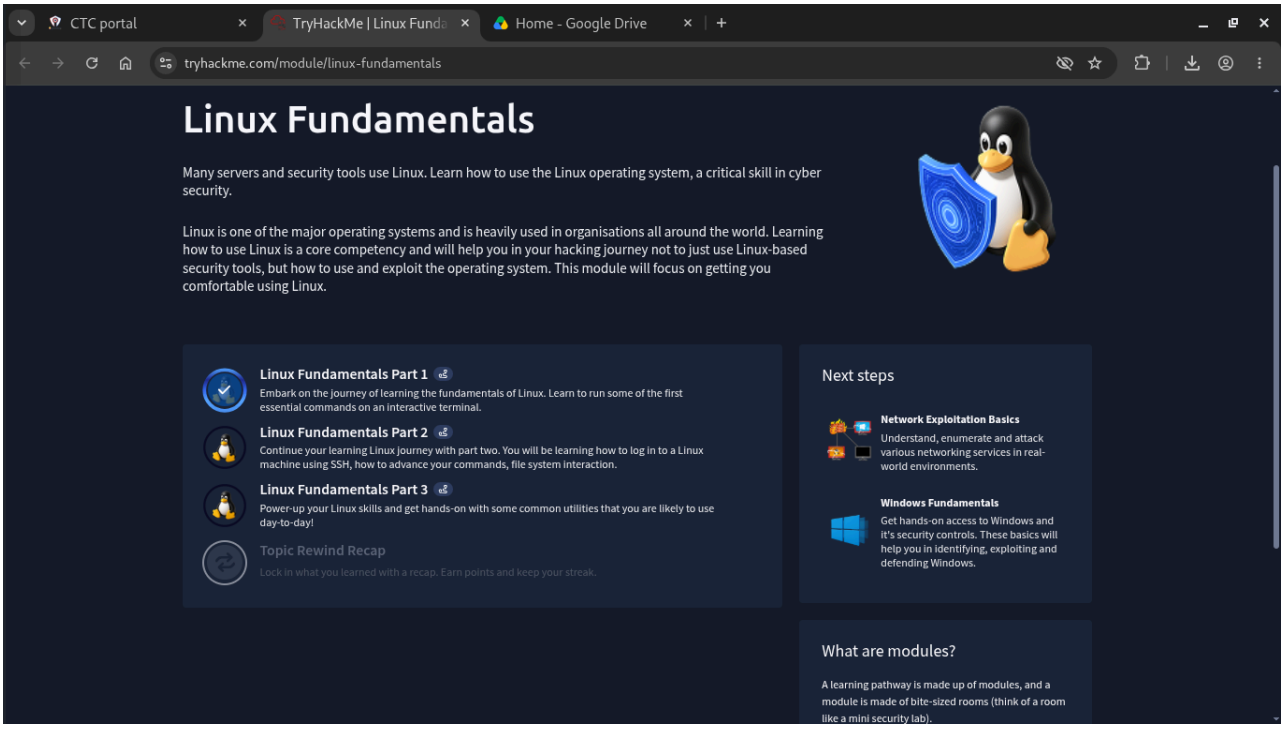
Linux Fundamentals

Name: Matyas Abraham

IDNo: CTC-207-26

Overview of the Room

The room I was assigned is TryHackMe's "Linux Fundamentals" module, which is actually split into three parts. The module page describes it as a way to get comfortable with Linux since so many servers and security tools run on it, and the whole point is to build the base skills I'll need before moving into things like network exploitation and privilege escalation later on. Below is a screenshot of the module landing page showing all three parts and my progress on them.



As the screenshot shows, I've fully completed Part 1 (it has the blue checkmark), and Parts 2 and 3 are the next rooms in the pathway. Most of this write-up focuses on Part 1 since that's the room I actually worked through this week — I've added a short summary of what Parts 2 and 3 will cover at the end.

Linux Fundamentals Part 1 — What I Did

Part 1 is done entirely in the browser using TryHackMe's built-in terminal, so there was nothing to install on my own machine. As soon as the machine deployed I was dropped into an Ubuntu 20.04.6 LTS box and given a target IP (10.130.179.229) to work from. The room is broken into five tasks, and I'll go through each one and explain what it actually taught me, not just what it says on the page.

Task 1 — Introduction

This task is mostly context-setting. It explains why Linux matters in cybersecurity — a huge share of servers,

routers, and the security tools we'll use later (like Nmap, Metasploit, Wireshark) either run on Linux or were built assuming you know your way around a Linux shell. It also walks through connecting to the target machine and confirming the terminal is working. Nothing technical to screenshot here, it's really just orientation before the real commands start.

Task 2 — Talking to Linux

This is where I actually started typing commands. The task introduces the terminal itself and the idea that everything in Linux, even things with a graphical interface, can usually be done faster and more precisely through the command line. I practiced basic commands like whoami to confirm which user I was logged in as, and got introduced to the general syntax pattern that almost every Linux command follows:

command [options/flags] [arguments]

For example, running a command with -h or --help at the end almost always prints a usage guide, which is genuinely one of the more useful habits I picked up here — instead of guessing or searching online every time, I can just ask the tool itself what it needs.

Task 3 — Finding Your Way Around

This task covers navigating the Linux filesystem, which felt the most "foundational" out of everything in the room. I worked with:

- pwd — prints the full path of the directory I'm currently sitting in.
- ls (and ls -la) — lists the contents of a directory, with -la showing hidden files and detailed permissions/ownership info.
- cd — changes directory, including shortcuts like cd .. to go up one level and cd ~ to jump straight back to the home directory.

What clicked for me here is that Linux treats basically everything as a file inside one single tree structure starting at root (/), which is very different from how Windows splits things across drive letters like C:\ and D:\. Once I understood that the whole filesystem is one tree, navigating with relative paths (like ../folder) versus absolute paths (like /home/user/folder) made a lot more sense.

Task 4 — Let the Machine Do the Searching

This task was about not manually hunting through folders when the machine can search for me. The main commands were:

- find — searches for files and directories based on name, type, size, or modification time, e.g. find / -name "*.txt".
- locate — searches a prebuilt index of the filesystem, which makes it much faster than find, though the index has to be updated (updatedb) to catch recently created files.
- grep — searches inside the contents of files for a matching pattern, e.g. grep "root" /etc/passwd.

This was probably my favorite task in the room because it's directly useful for real security work — a lot of CTF challenges and even real penetration tests come down to finding one specific file or one specific string buried somewhere in a large filesystem, and doing that by hand would take forever.

Task 5 — Shell Operators (Combining Commands)

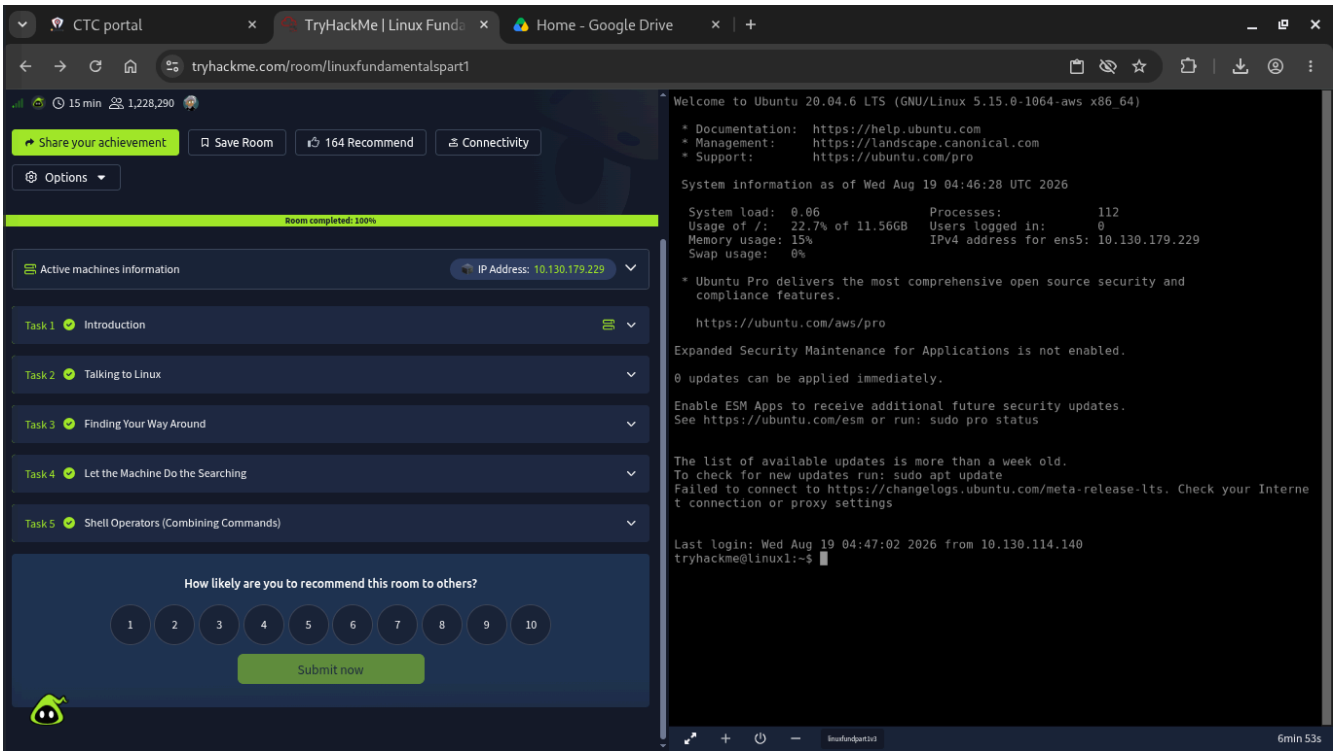
The last task covers chaining commands together instead of running them one at a time. I learned:

- ; — runs multiple commands one after another regardless of whether the first one succeeded.
- && — runs the second command only if the first one succeeded (exit code 0), which is what I'd actually want most of the time.
- | (pipe) — takes the output of one command and feeds it in as the input to the next, e.g. cat file.txt | grep "error". • > and >> — redirect a command's output into a file, either overwriting it (>) or appending to it (>>).

Piping was the concept that took the longest to really sink in, but once it did, I could see how it turns a bunch of small, single-purpose commands into a much more powerful one-liner — which is basically the whole philosophy behind the Linux command line.

Room Completion

After finishing all five tasks I got a 100% completion bar and the "Share your achievement" option unlocked, which confirmed the room was fully done. Screenshot below:



Challenge Questions (In My Own Words)

Q: Why is learning Linux important for cybersecurity?

A: Because so much of the infrastructure we'll be attacking or defending — servers, routers, IoT devices, and most security tools themselves — runs on Linux. If I can't move around a Linux system comfortably, I'll be slow and make mistakes even before I get to the actual security work.

Q: What is the difference between an absolute path and a relative path?

A: An absolute path always starts from the root of the filesystem (/) and describes the full location of a file no matter where I currently am, like /home/maty/notes.txt. A relative path is based on my current location, so something like ../notes.txt only makes sense relative to whatever directory I happen to be sitting in at

that moment.

Q: What's the difference between find and locate?

A: find actively searches through the real filesystem at the moment I run it, so it's always accurate but can be slow on a big system. locate instead searches a pre-built database/index of the filesystem, so it's much faster, but that index needs to be refreshed with updatedb or it won't know about files created very recently.

Q: What does the pipe operator (|) do, and why is it useful?

A: It takes the output that one command would normally print to the screen and instead sends it straight into the next command as input. It's useful because it lets me combine small, simple commands into one longer chain that does something more complex, instead of saving output to a file and re-reading it every step.

Q: When would I use && instead of ; between two commands?

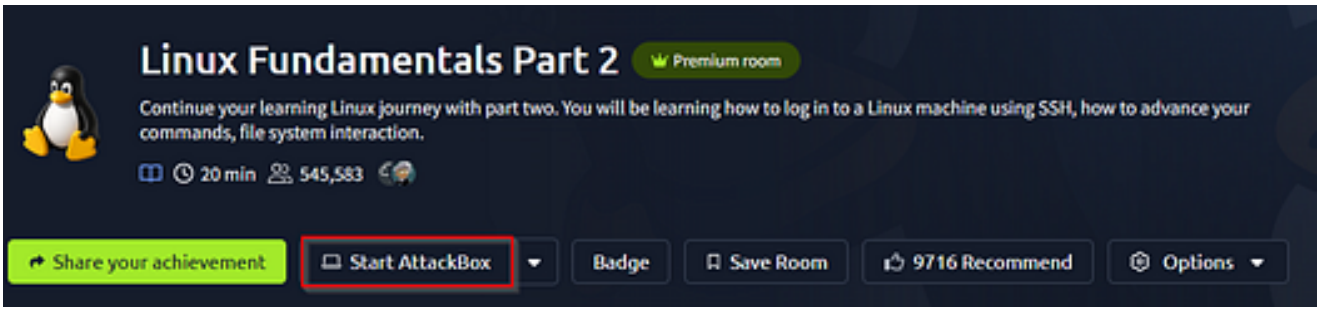
A: I'd use ; when I want both commands to run no matter what, even if the first one fails. I'd use && when the second command genuinely depends on the first one succeeding — for example, only extracting a file if the download actually completed successfully.

Linux Fundamentals Part 2 & 3 — Quick Overview

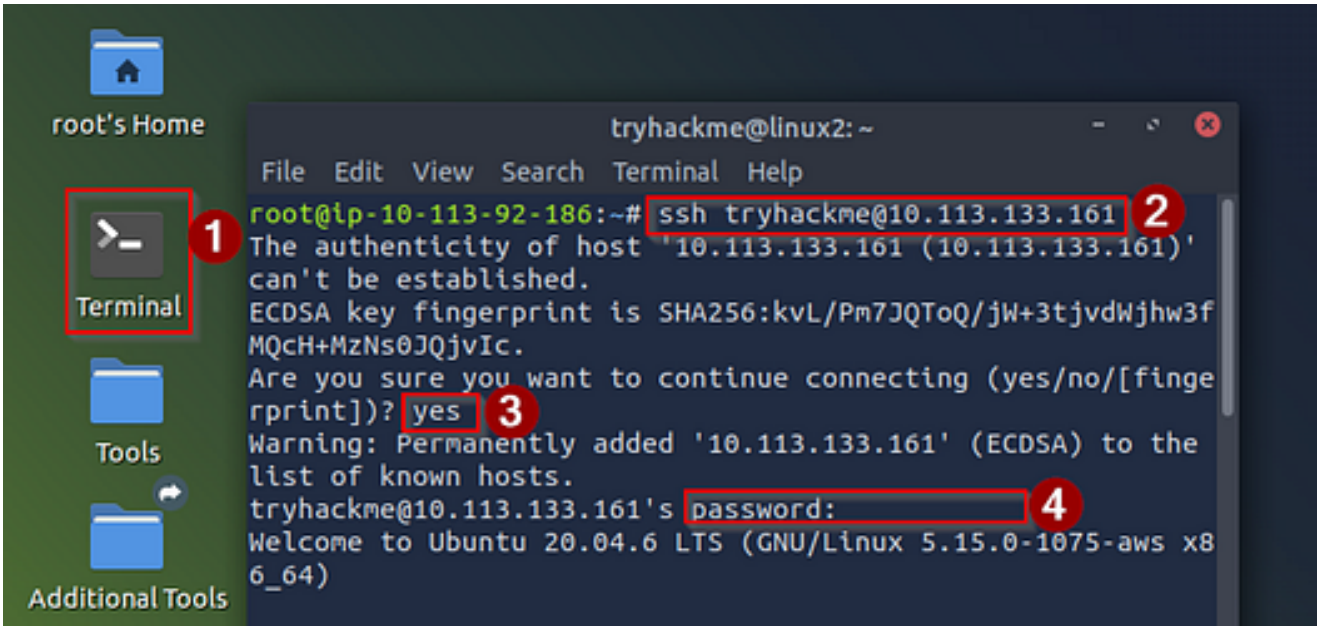
In Part 2, I explored connecting to a Linux machine using SSH, working more efficiently with commands, understanding file permissions, and navigating key system directories.

Accessing Your Linux Machine Using SSH (Deploy)

To access our Linux machine using SSH, we need to start the machine under the “Deploying Your Linux Machine” section, as shown in my screenshot above. After starting the machine, you go towards the Top of the page to find the info (Title and IP Address) about your machine. We will copy the IP address there as we will need it shortly.



Next up, we go to the Top of the page and click on Start AttackBox. This will give us access to the Linux Terminal, where we can connect to the machine using the SSH protocol.



After starting the AttackBox, open the terminal on the Desktop, then run the command to connect to the machine using SSH and the machine IP we copied a while ago. Eg.,`ssh tryhackme@10.113.133.161` Then type yes to confirm the connection and input the password, which, according to the room, is “tryhackme”.

Part 3

Terminal Text Editor

Text editors allow you to modify configuration files directly from the command line.

- **Nano**: A simple, modeless editor. Commands are visible at the bottom of the screen (e.g., `Ctrl + O` to save, `Ctrl + X` to exit).
- **Vim**: A powerful, modal editor. It starts in **Command Mode**. Press `i` for **Insert Mode** (to type), `Esc` to return to Command Mode, and type `:wq` to save and quit. [1]

2. General Utilities

- **`wget <URL>`**: Downloads files directly from the internet to your current directory.
- **`curl <URL>`**: Transfers data to or from a server; often used to test web connections or download files (`curl -O <URL>`).
- **`python3 -m http.server`**: Instantly starts a lightweight web server in your current directory for quick file sharing.

3. Process Management

Every running program in Linux is a "process" assigned a unique **Process ID (PID)**.

- **`ps aux`**: Lists every running process on the system, showing the user, memory, and CPU usage.
- **`top` / `htop`**: Displays real-time, interactive system resource usage and running processes.
- **`kill <PID>`**: Sends a termination signal to safely stop a running process.
- **`systemctl <status/start/stop/restart> <service>`**: Controls background system services (daemons) like web servers or SSH.

4. Automation with Cron

Cron is a time-based job scheduler used to automate repetitive tasks like backups or updates.

- **`crontab -e`**: Edits the automation schedule for your user.

- **Syntax**: Uses 5 fields to define time: `* * * * * <command>`
    - `Minute Hour Day of Month Month Day of Week`
    - *Example*: `0 5 * * * /backup.sh` runs the script every day at 5:00 AM.

5. Package Management (APT)

Advanced Package Tool (APT) manages software installation on Debian/Ubuntu-based systems.

- **`sudo apt update`**: Refreshes the local database of available software packages.
- **`sudo apt upgrade`**: Installs the latest available security updates for all current software.
- **`sudo apt install <package>`**: Downloads and installs a new program along with its dependencies.

6. System Logs

Logs are critical for troubleshooting errors and monitoring security.

- **`/var/log/`**: The standard system directory where all log files reside.
- **`/var/log/auth.log`**: Tracks user logins, sudo usage, and authentication attempts.
- **`/var/log/syslog`**: Central repository for general system messages and global activity logs.

Reference

- Youtube
- Wiki
- Medium
- Gemini